

Object-Oriented Programming

Practice Midterm

Time allowed: 120 minutes

Total: 100 points

General instructions

- Questions 1–3 each give you **working but badly designed code**. Your job is to *preserve the observable behaviour* while improving the design. A rewrite that changes what the user sees on screen earns no marks.
- Follow Python accessibility conventions: internal instance variables take a single leading underscore (`_center_x`); expose them for reading through `@property`. Code outside a class must **never** touch an underscore attribute.
- Assume `import pygame`, `import math`, `import random` are available and that `WIDTH`, `HEIGHT` = 800, 600.
- Marks are awarded for design quality and justification, not only for running code.

The given base class (used by Questions 1 and 2)

```
import pygame

class Ball:
    def __init__(self, center_x, center_y, radius, speed, color):
        self._center_x = center_x
        self._center_y = center_y
        self._radius = radius
        self._speed = speed
        self._color = color

    @property
    def center_x(self):
        return self._center_x

    @property
    def center_y(self):
        return self._center_y

    @property
    def radius(self):
        return self._radius

    @property
    def speed(self):
        return self._speed

    @property
    def color(self):
        return self._color

    def draw(self, screen):
        pygame.draw.circle(screen, self._color,
                           (int(self._center_x), int(self._center_y)),
                           self._radius)
```

```
def move(self):
    pass # A plain Ball does not move. Subclasses decide how.
```

Question 1 Refactor procedural code into a subclass — *BouncingBall* (25 points)

The program below works correctly: it animates three balls that bounce inside the window and change to a random colour on every wall hit. It is written entirely with global state and parallel dictionaries.

```
1 import math, random, pygame
2
3 WIDTH, HEIGHT, FPS = 800, 600, 60
4
5 balls = []
6
7 def random_color():
8     return (random.randint(50, 255), random.randint(50, 255),
9             random.randint(50, 255))
10
11 def create_ball(cx, cy, r, speed):
12     global balls
13     angle = random.uniform(0, 2 * math.pi)
14     balls.append({"x": cx, "y": cy, "r": r, "speed": speed,
15                 "dx": speed * math.cos(angle),
16                 "dy": speed * math.sin(angle),
17                 "color": random_color()})
18
19 def update_balls():
20     global balls
21     for b in balls:
22         b["x"] += b["dx"]
23         b["y"] += b["dy"]
24         hit = False
25         if b["x"] - b["r"] <= 0:
26             b["x"] = b["r"]; b["dx"] = abs(b["dx"]); hit = True
27         elif b["x"] + b["r"] >= WIDTH:
28             b["x"] = WIDTH - b["r"]; b["dx"] = -abs(b["dx"]); hit = True
29         if b["y"] - b["r"] <= 0:
30             b["y"] = b["r"]; b["dy"] = abs(b["dy"]); hit = True
31         elif b["y"] + b["r"] >= HEIGHT:
32             b["y"] = HEIGHT - b["r"]; b["dy"] = -abs(b["dy"]); hit = True
33         if hit:
34             b["color"] = random_color()
35
36 def draw_balls(screen):
37     global balls
38     for b in balls:
39         pygame.draw.circle(screen, b["color"],
40                             (int(b["x"]), int(b["y"])), b["r"])
```

Refactor the code into a class `BouncingBall` that **inherits** from the given `Ball`. Requirements:

- constructor signature (`center_x`, `center_y`, `radius`, `speed`) — **no colour argument**: the initial colour and the initial direction are randomised inside;
- the random direction must produce a motion of magnitude exactly `speed` px/frame;
- `move()` is overridden; `draw()` is **not** redefined;
- no `global` statement anywhere.
- rewrite the client code (creation loop, update loop, draw loop) using your class.

Question 2 *Replace a type code with subclasses — motion variants***(25 points)**

A teammate needed two more kinds of ball. Instead of writing subclasses, they added a kind string to a single class:

```

1  class Ball:
2      def __init__(self, kind, radius, speed, color,
3                  orbit_radius=0, start_angle=0.0):
4          self._kind = kind # "circular" or "horizontal"
5          self._radius = radius
6          self._speed = speed
7          self._color = color
8          self._orbit_radius = orbit_radius # unused when horizontal
9          self._angle = start_angle # unused when horizontal
10         self._orbit_center_x = WIDTH / 2 # unused when horizontal
11         self._orbit_center_y = HEIGHT / 2 # unused when horizontal
12
13         if kind == "circular":
14             self._center_x = self._orbit_center_x + \
15                 orbit_radius * math.cos(start_angle)
16             self._center_y = self._orbit_center_y + \
17                 orbit_radius * math.sin(start_angle)
18         elif kind == "horizontal":
19             self._center_x = -radius # enters from the left wall
20             self._center_y = HEIGHT / 2
21         else:
22             raise ValueError("unknown kind: " + kind)
23
24     @property
25     def center_x(self):
26         return self._center_x
27
28     @property
29     def center_y(self):
30         return self._center_y
31
32     def move(self):
33         if self._kind == "circular":
34             self._angle += self._speed / self._orbit_radius
35             self._angle %= 2 * math.pi
36             self._center_x = self._orbit_center_x + \
37                 self._orbit_radius * math.cos(self._angle)
38             self._center_y = self._orbit_center_y + \
39                 self._orbit_radius * math.sin(self._angle)
40         elif self._kind == "horizontal":
41             self._center_x += self._speed
42             if self._center_x - self._radius > WIDTH:
43                 self._center_x = -self._radius
44
45     def draw(self, screen):
46         if self._kind == "circular":
47             pygame.draw.circle(screen, self._color,
48                               (int(self._center_x), int(self._center_y)),
49                               self._radius)
50         elif self._kind == "horizontal":
51             pygame.draw.circle(screen, self._color,
52                               (int(self._center_x), int(self._center_y)),
53                               self._radius)

```

(a) Refactor the "circular" behaviour into a class `CircularBall` that extends the `Ball`

base class given on page 1. Constructor: (radius, speed, color, orbit_radius, start_angle=0.0). The starting centre must be computed from the orbit, not passed in. **Do not** redefine draw().

- (b) Refactor the "horizontal" behaviour into HorizontalBall. Constructor: (radius, speed, color). It enters from the middle of the left wall, travels right, and re-enters from the left only once it has *completely* left through the right wall.

Question 3 *Replace a control flag with composition — Controller* **(35 points)**

A third teammate needed a player-controlled ball and an enemy that chases it. They again reached for a flag — this time control:

```

1  class Ball:
2      def __init__(self, control, center_x, center_y, radius, speed, color,
3          target=None):
4          self._control = control # "keyboard" or "ai"
5          self._target = target # used only when control == "ai"
6          self._center_x = center_x
7          self._center_y = center_y
8          self._radius = radius
9          self._speed = speed
10         self._color = color
11
12         @property
13         def center_x(self):
14             return self._center_x
15
16         @property
17         def center_y(self):
18             return self._center_y
19
20         def move(self):
21             if self._control == "keyboard":
22                 keys = pygame.key.get_pressed()
23                 if keys[pygame.K_LEFT] or keys[pygame.K_a]:
24                     self._center_x -= self._speed
25                 if keys[pygame.K_RIGHT] or keys[pygame.K_d]:
26                     self._center_x += self._speed
27                 if keys[pygame.K_UP] or keys[pygame.K_w]:
28                     self._center_y -= self._speed
29                 if keys[pygame.K_DOWN] or keys[pygame.K_s]:
30                     self._center_y += self._speed
31             elif self._control == "ai":
32                 if self._target._center_x > self._center_x:
33                     self._center_x += self._speed
34                 elif self._target._center_x < self._center_x:
35                     self._center_x -= self._speed
36                 if self._target._center_y > self._center_y:
37                     self._center_y += self._speed
38                 elif self._target._center_y < self._center_y:
39                     self._center_y -= self._speed
40
41         def draw(self, screen):
42             pygame.draw.circle(screen, self._color,
43                 (int(self._center_x), int(self._center_y)),
44                 self._radius)

```

This time **do not** solve the problem with subclasses. Redesign it so that a Ball *has-a* Controller instead of *being-a* particular kind of ball.

- (a) Write the abstract base class `Controller` (read-only `x`, `y`, `speed` properties and a `move()` method) and the redesigned `Ball`. The `Ball.move()` must **delegate** to its controller and then **synchronise** its own centre with the controller's position. `Ball` must contain no movement logic and no control flag.
- (b) Write `KeyboardController`: WASD *and* arrow keys, `speed` pixels per frame, with the diagonal-speed bug fixed and the position kept inside the window.
- (c) Write `ChasingController(x, y, speed, target)`: it moves towards the target ball's current centre at exactly `speed` pixels per frame along the straight line to it, and stops cleanly when close enough. Then show the client code that wires a player ball and a chasing enemy together.

Question 4 *Integration and Design Justification*

(15 points)

Write a complete, runnable Pygame program that opens an 800×600 window at 60 FPS, handles the `QUIT` event, and animates all of the following in one scene: three `BouncingBall` objects (Q1), one `CircularBall` and one `HorizontalBall` (Q2), one keyboard-controlled ball and one ball chasing it (Q3). The game loop must treat every object uniformly — **no `isinstance` checks and no branching on object type**.

End of examination — total 100 points.